

On this page

*Last updated on **Aug 5, 2026***

Observability in Databricks for jobs, Lakeflow pipelines, and Lakeflow Connect

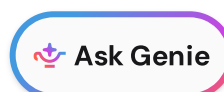
Monitoring your streaming applications' performance, cost, and health is essential to building reliable, efficient ETL pipelines. Databricks provides a rich set of observability features across Jobs, Lakeflow pipelines, and Lakeflow Connect to help diagnose bottlenecks, optimize performance, and manage resource usage and costs.

These best practices span the following areas:

- Key streaming performance metrics
- Event log schemas and example queries
- Streaming query monitoring
- Cost observability using system tables
- Exporting logs and metrics to external tools

Key metrics for streaming observability

When operating streaming pipelines, monitor the following key metrics:



Metric	Purpose	
Backpressure	Monitors the number of files and offsets (sizes). Helps identify bottlenecks and ensures the system can handle incoming data without falling behind.	
Throughput	Tracks the number of messages processed per micro-batch. Assess pipeline efficiency and check that it keeps pace with data ingestion.	
Duration	Measures the average duration of a micro-batch. Indicates processing speed and helps tune batch intervals.	
Latency	Indicates how many records/messages are processed over time. Helps understand end-to-end pipeline delays and optimize for lower latencies.	
Cluster utilization	Reflects CPU and memory usage (%). Ensures efficient resource use and helps scale clusters to meet processing demands.	
Network	Measures data transferred and received. Useful for identifying network bottlenecks and improving data transfer performance.	
Checkpoint	Identifies processed data and offsets. Ensures consistency and enables fault tolerance during failures.	
Cost	Shows a streaming application's hourly, daily, and monthly costs. Aids in budgeting and resource optimization.	
Lineage	Displays datasets and layers created in the streaming application. Facilitates data transformation, tracking, quality assurance, and debugging.	

Cluster logs and metrics

Databricks cluster logs and metrics provide detailed insights into cluster performance and utilization. These logs and metrics include information about CPU, memory, disk I/O, network traffic, and other system metrics. Monitoring these metrics is crucial for optimizing cluster performance, managing resources efficiently, and troubleshooting issues.

Databricks cluster logs and metrics offer detailed insights into cluster performance and resource utilization. These include CPU and memory usage, disk I/O, and network traffic. Monitoring these metrics is critical for:

- Optimizing cluster performance.
- Managing resources efficiently.
- Troubleshooting operational issues.

The metrics can be leveraged through the Databricks UI or exported to personal monitoring tools. See [Notebook example: Datadog metrics](#).

Spark UI

The Spark UI shows detailed information about the progress of jobs and stages, including the number of tasks completed, pending, and failed. This helps you understand the execution flow and identify bottlenecks.

For streaming applications, the [Streaming tab](#) shows metrics such as input rate, processing rate, and batch duration. It helps you monitor your streaming jobs' performance and identify any data ingestion or processing issues.

See [Debugging with the Spark UI](#) for more information.

Compute metrics

The compute metrics help you understand the cluster utilization. As your job runs, you can see how it scales and how your resources are affected. You'll be able to find memory pressure that could lead to OOM failures or CPU pressure that could cause long delays. Here are the specific metrics you'll see:

- **Server Load Distribution:** Each node's CPU utilization over the past minute.



- **CPU Utilization:** The percentage of time the CPU spent in various modes (for example, user, system, idle, and iowait).
- **Memory Utilization:** Total memory usage by each mode (for example, used, free, buffer, and cached).
- **Memory Swap Utilization:** Total memory swap usage.
- **Free Filesystem Space:** Total filesystem usage by each mount point.
- **Network Throughput:** The number of bytes received and transmitted through the network by each device.
- **Number of Active Nodes:** The number of active nodes at every timestamp for the given compute.

See [Hardware metric charts](#) for more information.

System tables

Cost monitoring

Databricks system tables provide a structured approach to monitor job cost and performance. These tables include:

- Job run details.
- Resource utilization.
- Associated costs.

Use these tables to understand operational health and financial impact.

Requirements

To use system tables for cost monitoring:

- An account admin must enable the `system.lakeflow` schema.
- Users must either:
 - Be both a metastore admin and an account admin, or
 - Have `USE` and `SELECT` permissions on the system schemas.



Example query: Most expensive jobs (last 30 days)

This query identifies the most expensive jobs over the past 30 days, aiding in cost analysis and optimization.

SQL

 Ask Genie

```

WITH list_cost_per_job AS (
  SELECT
    t1.workspace_id,
    t1.usage_metadata.job_id,
    COUNT(DISTINCT t1.usage_metadata.job_run_id) AS runs,
    SUM(t1.usage_quantity * list_prices.pricing.default) AS list_cost,
    FIRST(identity_metadata.run_as, true) AS run_as,
    FIRST(t1.custom_tags, true) AS custom_tags,
    MAX(t1.usage_end_time) AS last_seen_date
  FROM system.billing.usage t1
  INNER JOIN system.billing.list_prices list_prices ON
    t1.cloud = list_prices.cloud AND
    t1.sku_name = list_prices.sku_name AND
    t1.usage_start_time >= list_prices.price_start_time AND
    (t1.usage_end_time <= list_prices.price_end_time OR
list_prices.price_end_time IS NULL)
  WHERE
    t1.billing_origin_product = "JOBS"
    AND t1.usage_date >= CURRENT_DATE() - INTERVAL 30 DAY
  GROUP BY ALL
),
most_recent_jobs AS (
  SELECT
    *,
    ROW_NUMBER() OVER(PARTITION BY workspace_id, job_id ORDER BY
change_time DESC) AS rn
  FROM
    system.lakeflow.jobs QUALIFY rn=1
)
SELECT
  t2.name,
  t1.job_id,
  t1.workspace_id,
  t1.runs,
  t1.run_as,
  SUM(list_cost) AS list_cost,
  t1.last_seen_date
FROM list_cost_per_job t1
LEFT JOIN most_recent_jobs t2 USING (workspace_id, job_id)
GROUP BY ALL
ORDER BY list_cost DESC

```

Lakeflow pipelines

The Lakeflow pipelines event log captures a comprehensive record of all pipeline events, including:

- Audit logs.
- Data quality checks.
- Pipeline progress.
- Data lineage.

The event log is automatically enabled for all Lakeflow pipelines and can be accessed via:

- **Pipeline UI:** View logs directly.
- **Pipelines API:** Programmatic access.
- **Direct query:** Query the event log table.

For more information, see [event log schema for Lakeflow pipelines](#).

Example queries

These example queries help monitor the performance and health of pipelines by providing key metrics such as batch duration, throughput, backpressure, and resource utilization.

Each query reads from `event_log_raw`, a view over your pipeline's event log table. To create it, see [Query the event log](#).

Average batch duration

This query calculates the average duration of batches processed by the pipeline.

SQL



```

SELECT
  (max_t - min_t) / batch_count as avg_batch_duration_seconds,
  batch_count,
  min_t,
  max_t,
  date_hr,
  message
FROM
  -- /60 for minutes
  (
    SELECT
      count(*) as batch_count,
      unix_timestamp(
        min(timestamp)
      ) as min_t,
      unix_timestamp(
        max(timestamp)
      ) as max_t,
      date_format(timestamp, 'yyyy-MM-dd:HH') as date_hr,
      message
    FROM
      event_log_raw
    WHERE
      event_type = 'flow_progress'
      AND level = 'METRICS'
    GROUP BY
      date_hr,
      message
  )
ORDER BY
  date_hr desc

```

Average throughput

This query calculates the average throughput of the pipeline in terms of processed rows per second.

SQL


```

SELECT
  (max_t - min_t) / total_rows as avg_throughput_rps,
  total_rows,
  min_t,
  max_t,
  date_hr,
  message
FROM
  -- /60 for minutes
  (
    SELECT
      sum(
        details:flow_progress:metrics:num_output_rows
      ) as total_rows,
      unix_timestamp(
        min(timestamp)
      ) as min_t,
      unix_timestamp(
        max(timestamp)
      ) as max_t,
      date_format(timestamp, 'yyyy-MM-dd:HH') as date_hr,
      message
    FROM
      event_log_raw
    WHERE
      event_type = 'flow_progress'
      AND level = 'METRICS'
    GROUP BY
      date_hr,
      message
  )
ORDER BY
  date_hr desc

```

Backpressure

This query measures the pipeline's backpressure by checking the data backlog.

SQL

```
SELECT
  timestamp,
  DOUBLE(
    details:flow_progress:metrics:backlog_bytes
  ) AS backlog_bytes,
  DOUBLE(
    details:flow_progress:metrics:backlog_files
  ) AS backlog_files
FROM
  event_log_raw
WHERE
  event_type = 'flow_progress'
```

Cluster and slots utilization

This query has insights into the utilization of clusters or slots used by the pipeline. Only applies to classic compute clusters. See [Details for cluster_resources event](#).

SQL

```

SELECT
  date_trunc("hour", timestamp) AS hour,
  AVG (
    DOUBLE (
      details:cluster_resources:num_task_slots
    )
  ) AS num_task_slots,
  AVG (
    DOUBLE (
      details:cluster_resources:avg_num_task_slots
    )
  ) AS avg_num_task_slots,
  AVG (
    DOUBLE (
      details:cluster_resources:num_executors
    )
  ) AS num_executors,
  AVG (
    DOUBLE (
      details:cluster_resources:avg_task_slot_utilization
    )
  ) AS avg_utilization,
  AVG (
    DOUBLE (
      details:cluster_resources:avg_num_queued_tasks
    )
  ) AS queue_size
FROM
  event_log_raw
WHERE
  details : cluster_resources : avg_num_queued_tasks IS NOT NULL
  AND origin.update_id = (
    SELECT origin.update_id
    FROM event_log_raw
    WHERE event_type = 'create_update'
    ORDER BY timestamp DESC
    LIMIT 1
  )
GROUP BY
  1;

```

Jobs

You can monitor streaming queries in jobs through the Streaming Query Listener.  **Ask Genie**

Attach a listener to the Spark session to enable the Streaming Query Listener in Databricks. This listener monitors the progress and metrics of your streaming queries. It can be used to push metrics to external monitoring tools or log them for further analysis.

Example: Export metrics to external monitoring tools

NOTE

This is available in Databricks Runtime 11.3 LTS and above for Python and Scala.

You can export streaming metrics to external services for alerting or dashboarding by using the `StreamingQueryListener` interface.

Here is a basic example of how to implement a listener:

Python

```
from pyspark.sql.streaming import StreamingQueryListener

class MyListener(StreamingQueryListener):
    def onQueryStarted(self, event):
        print("Query started: ", event.id)

    def onQueryProgress(self, event):
        print("Query made progress: ", event.progress)

    def onQueryTerminated(self, event):
        print("Query terminated: ", event.id)

spark.streams.addListener(MyListener())
```

Example: Use query listener within Databricks

Below is an example of a `StreamingQueryListener` event log for a Kafka to Delta Lake streaming query:

JSON

 Ask Genie

```

{
  "id": "210f4746-7caa-4a51-bd08-87cabb45bdbe",
  "runId": "42a2f990-c463-4a9c-9aae-95d6990e63f4",
  "timestamp": "2024-05-15T21:57:50.782Z",
  "batchId": 0,
  "batchDuration": 3601,
  "numInputRows": 20,
  "inputRowsPerSecond": 0.0,
  "processedRowsPerSecond": 5.55401277422938,
  "durationMs": {
    "addBatch": 1544,
    "commitBatch": 686,
    "commitOffsets": 27,
    "getBatch": 12,
    "latestOffset": 577,
    "queryPlanning": 105,
    "triggerExecution": 3600,
    "walCommit": 34
  },
  "stateOperators": [
    {
      "operatorName": "symmetricHashJoin",
      "numRowsTotal": 20,
      "numRowsUpdated": 20,
      "allUpdatesTimeMs": 473,
      "numRowsRemoved": 0,
      "allRemovalsTimeMs": 0,
      "commitTimeMs": 277,
      "memoryUsedBytes": 13120,
      "numRowsDroppedByWatermark": 0,
      "numShufflePartitions": 5,
      "numStateStoreInstances": 20,
      "customMetrics": {
        "loadedMapCacheHitCount": 0,
        "loadedMapCacheMissCount": 0,
        "stateOnCurrentVersionSizeBytes": 5280
      }
    }
  ],
  "sources": [
    {
      "description": "KafkaV2[Subscribe[topic-1]]",
      "numInputRows": 10,
      "inputRowsPerSecond": 0.0,
      "processedRowsPerSecond": 2.77700638711469,
      "metrics": {
        "avgOffsetsBehindLatest": "0.0",
        "estimatedTotalBytesBehindLatest": "0.0",

```

```

        "maxOffsetsBehindLatest": "0",
        "minOffsetsBehindLatest": "0"
    },
    {
        "description": "DeltaSource[file:/tmp/spark-1b7cb042-bab8-4469-bb2f-733c15141081]",
        "numInputRows": 10,
        "inputRowsPerSecond": 0.0,
        "processedRowsPerSecond": 2.77700638711469,
        "metrics": {
            "numBytesOutstanding": "0",
            "numFilesOutstanding": "0"
        }
    }
]
}

```

For more examples, see: [Examples](#).

Query progress metrics

Query progress metrics are essential for monitoring the performance and health of your streaming queries. These metrics include the number of input rows, processing rates, and various durations related to the query execution. You can observe these metrics by attaching a `StreamingQueryListener` to the Spark session. The listener emits events containing these metrics at the end of each streaming epoch.

For example, you can access metrics using the

`StreamingQueryProgress.observedMetrics` map in the listener's `onQueryProgress` method. This allows you to track and analyze the performance of your streaming queries in real-time.

Python

```

class MyListener(StreamingQueryListener):
    def onQueryProgress(self, event):
        print("Query made progress: ", event.progress.observedMetrics)

```

 Ask Genie

On this page

Was this page helpful?

☐ Yes

☐ No

[Privacy Notice](#) 

☐ Send feedback

[Terms of Use](#) 

[Modern Slavery Statement](#) 

[California Privacy](#) 

[Your Privacy Choices](#)  

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark and the Spark logo are trademarks of the Apache Software Foundation.

 Ask Genie